

A simply typed λ -calculus in one page

Hans

October 27, 2022

In this note I present a type system for a version the applied λ -calculus from the previous note. In this version I will leave out the **lecrec**-construct (i.e. there no recursion). This means that the formation rules for expressions $e \in \mathbf{Exp}$ will be

$$e ::= x \mid \lambda x: \mathbf{t}. e_1 \mid e_1 e_2 \mid \text{let } x: \mathbf{t} = e_1 \text{ in } e_2 \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \mid c \quad (1)$$

where $c \in \lambda\text{-}\mathbf{Con}$, where we let $\lambda\text{-}\mathbf{Con} = \{\mathbf{n} \mid n \in \mathbb{Z}\} \cup \{\mathbf{True}, \mathbf{False}\} \cup \{\mathbf{plus}, \mathbf{minus}, \mathbf{and}, \mathbf{or}, \mathbf{not}, \mathbf{=}\}$. As one can see from (1), the variable binders in abstractions and local declarations are now annotated with types.¹

The set of types **Types** is given by the following formation rules:

$$t ::= \mathbf{Int} \mid \mathbf{Bool} \mid t_1 \rightarrow t_2 \mid t_1 \times t_2.$$

In order to type an expression, we must know the types of its variables, since expressions can contain variables. A *type environment* E provides us with this information. It is a partial function

$$E : \mathbf{Var} \rightarrow \mathbf{Types}$$

that tells us what the types of variables are assumed to be. If $E(x) = t$, this means that variable x has type t . We write $E, x : t$ to denote the environment that extends E with a type binding that says that the type of x is t .

We also need a constant environment $\Sigma : \lambda\text{-}\mathbf{Con} \rightarrow \mathbf{Types}$ that tells us what the types of constants are. Our type judgements are of the form $E \vdash e : t$, which should be read as saying that, given the type environment E , the expression e has type t . Note that Σ is always the same environment, so we do not mention it explicitly in type judgements.

A type judgement is valid if we can build a finite derivation tree for it using the *type rules* given below, in such a way that the leaves of the tree are instances of (VAR) and (CONST).

$$\begin{array}{ll} \text{(VAR)} & \frac{E(x) = t}{E \vdash x : t} \\ \text{(CONST)} & E \vdash c : t \text{ if } \Sigma(c) = t \\ \text{(APP)} & \frac{E \vdash e_1 : t_1 \rightarrow t_2 \quad E \vdash e_2 : t_1}{E \vdash e_1 e_2 : t_2} \\ \text{(ABS)} & \frac{E, x : t_1 \vdash e : t_2}{E \vdash \lambda x: \mathbf{t}_1. e : t_1 \rightarrow t_2} \\ \text{(IF)} & \frac{E \vdash e_0 : \mathbf{Bool} \quad E \vdash e_1 : t \quad E \vdash e_2 : t}{E \vdash \text{if } e_0 \text{ then } e_1 \text{ else } e_2 : t} \\ \text{(LET)} & \frac{E \vdash e_1 : t_1 \quad E, x : t_1 \vdash e_2 : t_2}{E \vdash \text{let } x: \mathbf{t}_1 = e_1 \text{ in } e_2 : t_2} \end{array}$$

A very important rule in any type system is (VAR) since it tells us how to find the type of a variable. The rule is simple: We simply find the type of x by looking it up in the environment. In the rules (ABS), (LET) and (LETREC) we add assumptions to the type environment in the premise of the rule that tell us what the type of the new bound variable x must be.

Our type system is *sound*: If an expression has a type t and can reduce, the resulting expression has the same type; this is called the *subject reduction* property. Moreover, if e is well-typed, it will eventually reduce to a value v (written $e \Rightarrow^* v$); this is called the *strong normalization* property.

Theorem 1 (Type soundness). *Suppose $E \vdash e : t$. Then we have that*

1. *If $e \Rightarrow e'$ then we also have that $E \vdash e' : t$.*
2. *e will eventually reduce to a value, that is, for some value v we have that $e \Rightarrow^* v$.*

(If we allowed for recursion, the strong normalization property would not hold.)

¹In presentations of the λ -calculus to use a single colon $:$ for type annotations; Haskell uses two colons $::$ to distinguish this from the list constructor 'cons'.